

2-Processes



Outline

- Multiprogramming
- Process Definition
- Process Object
- UNIX Processes
 - Operations on processes
 - Signals

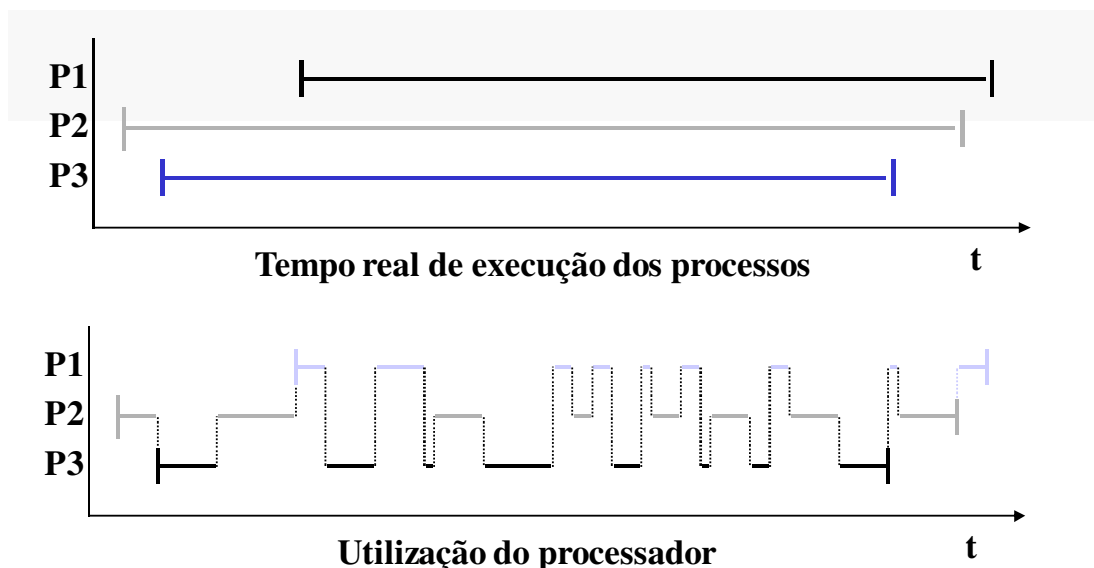


Multiprogramming

- “Parallel” execution of multiple programs in the same computer
- Each instance of a running program is called a process
- Looking at the process execution with a fine detail the parallelism is not real
- Pseudo-parallelism - implementing a multiprogrammed system over a computer with a single processor

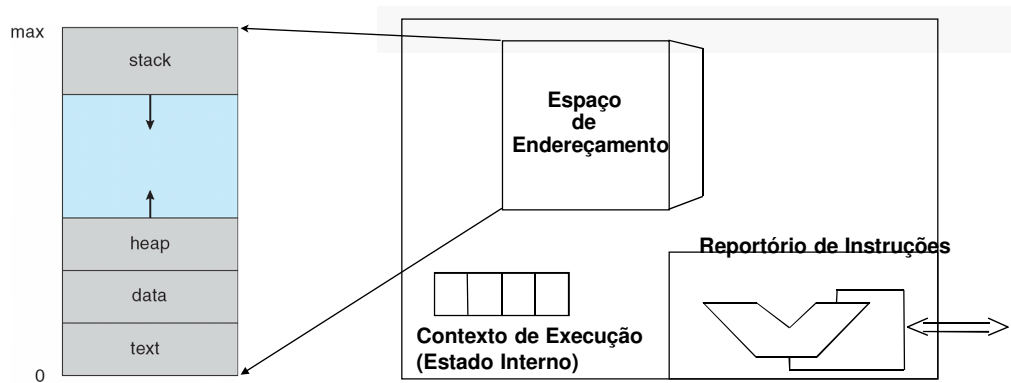


Pseudo - parallelism



Process Concept

- Process as a virtual machine



Main resources made available to a process by the operating system



Process control Block

- Fields

- Process ID
- Program in execution
- Addressing Space
- CPU Registry values
- Program Counter
- Process Priority
- Father process
- Process descriptors
- Computer resource quota usage
- Security context

- Operations over processes

- Create
- Terminate
- Wait for the termination of a child process.



UNIX processes

- Process Identification

- Integer value named PID
- Some are pre assigned by the SO: process 0 is the *swapper* (memory management) and process 1 *init* performs the system startup

- We have a tree of Processes in UNIX

- A process inherits its context from its father process
- Each process knows its father.
- When the father process ends while their children are still running they are adopted by the init process (PID=1).

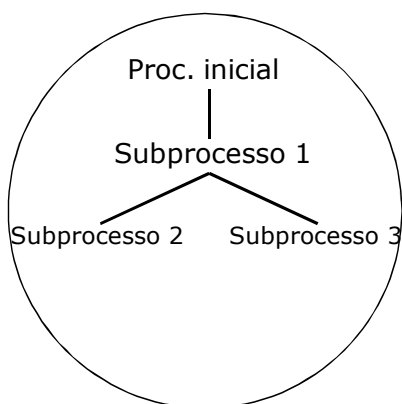
- Process have different execution priorities.

- We will study the process scheduling algorithms later.

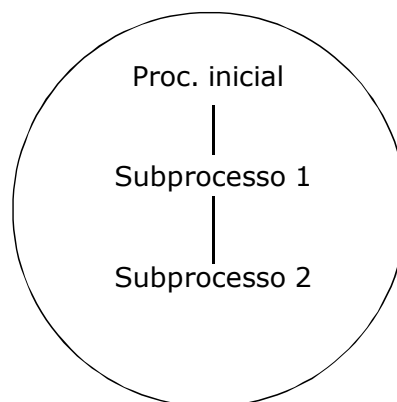


Process Hierarchy

Utilizador A



Utilizador B



Some information are inherited



UNIX Processes

- Each process has an execution context with several environment variables useful for the application.

Example:
HOME=/Users/pmsobral
SHELL=/bin/bash
USER=pmsobral
PATH=/usr/local/bin:/bin:/usr/bin...

- This execution context is inherited from the father process and can be freely changed by the process.
- In C programs it is available through the envp parameter of the main function or through an external variable:

```
main (argc, argv, envp)
int argc, extern char **argv, **envp
```



Process Creation

`id = fork()`

The function has no parameters, in particular the file to execute. The image of the new process is a copy of the creator's.

The function returns the PID of the process. This parameter assumes different values according to the process in which the return takes place:

- To the parent process is returned the "pid" of the child**
- To the child process is returned 0(zero)**
- 1 is returned in the event of an error**

This behavior is not present in the sequential programming.



Exemplo de fork()

```
main() {  
    int pid;  
  
    pid = fork();  
    if (pid == 0) {  
  
        /* código do processo filho */  
  
    } else {  
  
        /* código do processo pai */  
  
    }  
  
    /* instruções seguintes */  
}
```



Process Termination

- Terminates the process and frees all the resources used by it to the OS
- Notifies the father process about it's child termination

`void exit (int status)`

status is a parameter that allows the father notification on the status of the terminating child.
Usually a negative value indicates an error condition.



Process Termination

- In UNIX there is a system call for the father to synchronize with a child termination
- The father process is suspended by the OS until one of its Childs terminates.

`int wait( int *status)`

Returns the PID of the terminating process. The father may be waiting for the termination of several processes

Returns the termination status of the child process as sent by the exit system call



UNIX Processes

```
main () {
    int pid, estado;

    pid = fork ();
    if (pid == 0) {
        /* algoritmo do processo filho */
        exit(0);
    } else {
        /* o processo pai bloqueia-se à espera da
           terminação do processo filho */
        pid = wait (&estado);
    }
}
```



Program Execution

```
int execl(char* ficheiro, arg0, arg1,..., argn,0)
```

```
int execv(char* ficheiro, *argv [])
```

Path to the executable file

Parameters to the new program. They can be sent as individual pointers or as an array of character pointers. These parameters are sent to the main function of the new program and are available in the argv argument.



exec() example

```
main ()
{
    int pid;

    pid = fork ();
    if (pid == 0) {
        execl ("/bin/who", "who", 0);
        /* controlo deveria ser transferido para o novo
           programa */
        printf ("Erro no execl\n");
        exit (-1);
    } else {
        /* algoritmo do proc. pai */
    }
}
```

Por convenção o arg0 é o nome do programa



Shell

- The shell is a good example of *fork* and *exec*

```
while (TRUE){
    prompt();
    read_command (command, params);

    pid = fork ();
    if (pid < 0) {
        printf ("Unable to fork"):
        continue;
    }
    if (pid !=0) {
        wait(&status)
    } else{
        execv (command, params):
    }
}
```

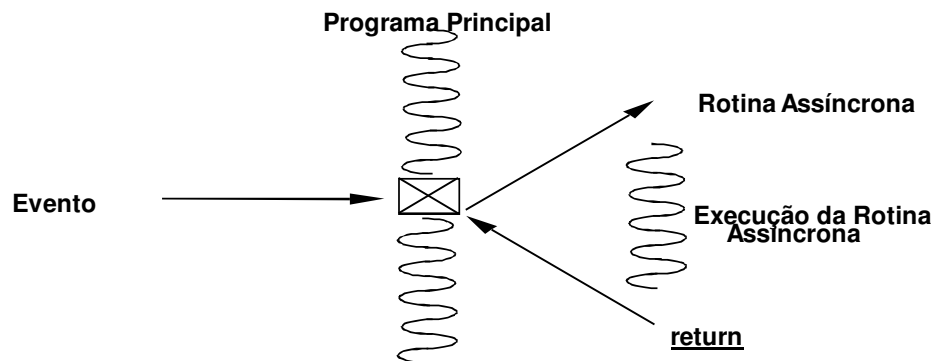


SIGNALS

- Certain events should be handled by applications however their occurrence is unpredictable.
 - Ex: Ctrl-C
 - Ex: *timeout*
- How can we handle this in sequential programming?
 - Asynchronous routines associated with each event.



Event Model



- Looks like another concept...



UNIX Signals

Signal	Cause
SIGALRM	timer expired
SIGFPE	Division by 0
SIGINT	The user interrupted the process. Usually Ctrl+C
SIGTSTP	The user paused the process execution Usually Ctrl+Z
SIGSTOP	Stop signal. Cannot be handled.
SIGKILL	Kill Signal. Cannot be handled.
SIGSEGV	Segmentation Violation
SIGTERM	The user asks the process to end
SIGUSR1	Defined by the user
SIGUSR2	Defined by the user



Signal Handling

- 3 Possibilities:
 - End the process.
 - Ignore the signal.
 - Some signals like SIGKILL cannot be ignored. why?
 - Execute the handler routine
 - this routine can be associated with the signal using the system call: `signal()`
- Each signal has a default behavior that can be terminate the process or ignore the signal.



System call `signal()`

```
void (*signal (int sig, void (*func)(int))) (int);
```

pointer to the function previously associated with the signal

signal to be handled

Pointer to the function macro:
•SIG_DFL – default action
•SIG_IGN – ignore the signal

parameter to the handler routine



Handling a signal

```
#include <stdio.h>
#include <signal.h>

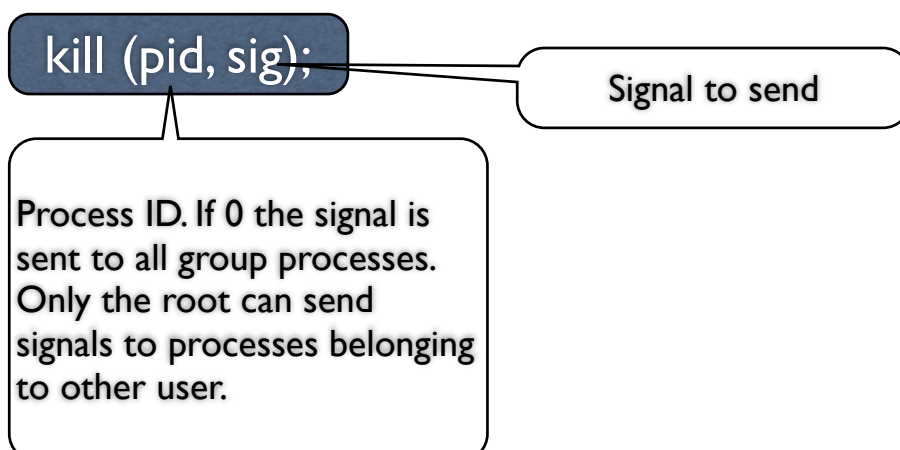
apanhaCTRLC () {
    char ch;
    printf ("Quer de facto terminar a execucao?\n");
    ch = getchar();
    if (ch == 's') exit(0);
    else {
        printf ("Entao vamos continuar\n");
        signal (SIGINT, apanhaCTRLC);
    }
}

main () {
    signal (SIGINT, apanhaCTRLC);
    printf("Associou uma rotina ao signal SIGINT\n");
    for (;;)
        sleep (10);
}
```



System call `kill()`

- Sends a signal to a process
- Tricky name. Why?



Other functions associated with signals

- **unsigned alarm (unsigned int segundos);**
 - The signal SIGALRM is sent to the process after the number of seconds. If the argument is zero the alarm is canceled.
- **pause();**
 - wait for a signal arrival
- **unsigned sleep (unsigned int segundos);**
 - The process sets an *alarm* e and is blocked waiting for the signal

